

第13章 模块与 Cargo 工程

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:熟悉 C 的头文件体系(#include、extern、static)、Makefile/CMake 构建,写过至少一个多文件 C 工程

🎯 本章学习目标

- 理解 C 头文件体系的痛点(重复包含、顺序依赖、宏污染)与 Rust 模块系统的答案
- 掌握模块三件套: `mod` 声明、`use` 导入、`pub` 可见性,理解"文件即模块"的映射
- 掌握模块树的组织:库 crate 与二进制 crate 的差异、目录模块的两种形态
- 掌握可见性体系:`pub` / `pub(crate)` / `pub(super)`,对照 C 的 `extern` 与 `static`
- 掌握 `use` 路径:绝对路径 `crate::`、相对 `super::`/`self::`,`pub use` 重导出与别名
- 掌握 Cargo 全流程:`new/build/run/test/check`,依赖管理与语义化版本
- 掌握 workspace 多 crate 工程与 crates.io 的发布/使用流程
- 能独立搭建一个"库 + 二进制 + 第三方依赖"的标准 Rust 工程

💡 从 C 到 Rust:本章主题如何迁移

C 的工程组织是"头文件 + 链接器"的二元体系:每个 .c 文件配一个 .h, #include 把声明文本复制进来;可见性靠 `extern`(跨文件)与 `static`(本文件)两个关键字;构建靠 Makefile/CMake 手工描述依赖图。痛处你早已熟悉:头文件重复包含(#pragma once 的补丁)、包含顺序敏感、宏在头文件间互相污染、忘了链接某个 .o 是链接器报"undefined reference"。这些问题的根源是同一个:"编译单元"与"逻辑模块"不是一回事,模块边界靠约定而非语言。

Rust 把工程组织收进语言与工具:模块(mod)是语言的正式公民,文件与模块一一对应,可见性(pub)由编译器检查;Cargo 是"构建系统 + 依赖管理 + 测试 + 文档"一体化的官方工具——你不再写 Makefile, Cargo.toml 声明依赖,锁文件(Cargo.lock)固定版本,一个命令下载构建全部依赖。本章先学模块的语言机制(§13.1-13.5),再学 Cargo 的工程流程(§13.6-13.7),最后实验搭一个"库 + CLI + 第三方依赖"的完整工程——这将是读者进入生产后每天都在用的骨架。

📦 前置知识自查

- C 的多文件工程:#include、头文件守卫、extern、static、链接错误(undefined reference)
- Makefile 或 CMakeLists.txt 的基本结构(至少见过 target/dependency 概念)
- 第10章的 crate/mod 概念引入、第12章的 Cargo.toml 与 features
- 第2章的基本语法(本章的示例代码需要能读懂)
- (可选)Python 的 import/package 或 Java 的 package,用于对照

本章学习路线

1. **第一阶段(约1.5小时):**§13.1–§13.3,模块机制与文件组织——先会"把代码拆进文件"
2. **第二阶段(约1.5小时):**§13.4–§13.5,可见性与 use 路径——模块系统的"接线"功夫
3. **第三阶段(约1.5小时):**§13.6–§13.7,Cargo 全流程与 workspace——工程落地的完整形态
4. **验收标准:**不看教程,能徒手新建 workspace、写库 crate、写 CLI、加依赖、跑测试

本章知识导图

- §13.1 头文件体系 vs 模块系统
- §13.2 模块基础:mod / use / pub
- §13.3 文件组织:文件即模块
- §13.4 可见性:pub 家族与 C 的对照
- §13.5 use 路径与重导出
- §13.6 Cargo 全流程:构建、测试、依赖
- §13.7 workspace 与 crates.io
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§13.1 头文件体系 vs 模块系统

13.1.1 C 的头文件之痛

```
// C:头文件是"文本复制"—每个 .c 编译单元各自复制一份声明
// math.h
#ifndef MATH_H                // 头文件守卫:手写的补丁
#define MATH_H
int add(int a, int b);
int sub(int a, int b);
#endif

// main.c
#include "math.h"              // 声明被复制进来
#include <stdio.h>
int main(void) {
    printf("%d\n", add(1, 2)); // 链接期才知道 add 在 math.c 里
    return 0;
}
// 问题:包含顺序敏感、宏互相污染、链接错误与声明不一致难以追踪
```

13.1.2 Rust 的模块:语言原生的边界

```
// Rust:模块是编译器的正式概念,不是文本复制
// src/main.rs(单文件演示;多文件见 §13.3)
mod math {                               // 声明模块:块内的代码属于 math
    pub fn add(a: i32, b: i32) -> i32 { a + b }
    pub fn sub(a: i32, b: i32) -> i32 { a - b }
}

fn main() {
    println!("{}", math::add(1, 2));    // 通过模块路径访问
}
```

C 的头文件体系	Rust 模块系统
#include 文本复制	mod 声明,模块是语言实体
#ifndef 守卫(手写补丁)	无需守卫:模块只编译一次
包含顺序敏感	声明顺序无关,模块树结构化
宏在头文件间污染	宏只在声明处与 use 处生效
extern:跨文件可见	pub:跨模块可见(显式)
static:本文件私有	默认私有(不写 pub 即私有)
链接期发现缺失符号	编译期发现缺失与未导出

§13.2 模块基础:mod / use / pub

13.2.1 mod 声明与默认私有

```
mod outer {                               // 声明模块 outer
    fn private() {}                       // 不写 pub:仅 outer 内可见
    pub fn public() {}                   // pub:对外可见

    pub mod inner {                       // 模块可以嵌套
        pub fn deep() {}
    }
}

fn main() {
    outer::public();                      // ✓ pub 可见
    // outer::private();                  // ERROR: private function(默认私有)
    outer::inner::deep();                 // ✓ 逐层 pub 才能到达
}
```

Rust 的可见性默认是全私有,需要显式 pub 开放——与 C 的默认全 extern(要 static 才收窄)完全相反。这个“默认私有”直接消灭了一整类“意外暴露内部符号”的问题。

13.2.2 use:把路径"接线"到本地

```
use std::collections::HashMap;           // 把完整路径绑定到短名字

fn main() {
    let mut m = HashMap::new();           // 不用写 std::collections::HashMap
    m.insert(1, "a");
    println!("{:?}", m);

    // 多路径一次性导入
    use std::io::{Read, Write};           // 花括号组
    // use std::fmt::{self, Debug}; // self 表示模块本身

    // 别名:解决命名冲突(对照 C 的 typedef 式缩名)
    use std::collections::HashMap as Map;
    let mut m2: Map<u32, u32> = Map::new();
    m2.insert(1, 2);
    println!("{:?}", m2);
}
```

§13.3 文件组织:文件即模块

13.3.1 单文件模块 → 拆成文件

```
// 规则一:src/main.rs(二进制 crate 根)或 src/lib.rs(库 crate 根)是模块树的根
// 规则二:mod math; 会在同目录找 math.rs 或 math/mod.rs

// ---- src/main.rs ----
mod math;           // 声明"math 模块在 src/math.rs"
mod utils;           // src/utils.rs

fn main() {
    println!("{}", math::add(1, 2));
    println!("{}", utils::greet());
}

// ---- src/math.rs ----
pub fn add(a: i32, b: i32) -> i32 { a + b }

// ---- src/utils.rs ----
pub fn greet() -> &'static str { "你好,模块世界" }
```

13.3.2 目录模块:子模块的两种形态

```
// 形态一:目录 + mod.rs(旧惯例,仍广泛使用)
// src/math/mod.rs 里声明 pub mod complex;
// src/math/complex.rs 是子模块文件
//
// 形态二:目录同名文件 + 子文件(新惯例,2021 edition 支持)
// src/math.rs 里声明 pub mod complex;
// src/math/complex.rs 作为子模块
//
// 两者等价;推荐形态二(与"文件即模块"的直觉一致)

// ---- src/lib.rs(库 crate 根:对外提供 API)----
pub mod math;           // 库的公共 API:math 模块对外可见
pub mod utils;

// ---- src/math.rs ----
pub mod complex;        // 子模块:complex.rs

// ---- src/math/complex.rs ----
pub fn norm(x: f64, y: f64) -> f64 {
    (x * x + y * y).sqrt()
}
```

⚠ 易错点 1:文件里的 pub 与"mod 声明处的 pub"是两回事

文件内函数写 pub 只代表"模块内可见性";模块能否被外部访问,取决于父模块里 mod 声明是否带 pub。即: mod math; 就算 math.rs 里全是 pub fn,外部 crate 也看不到 math——必须 pub mod math;。两层 pub 各管一层,漏掉任何一层都报"module is private"。

§13.4 可见性:pub 家族与 C 的对照

```
// 可见性四级(由宽到窄)
pub fn wide() {}           // 完全公开:本 crate 与所有依赖方可见
pub(crate) fn mid() {}     // 仅本 crate 内可见(二进制/库内部)
pub(super) fn narrow() {}  // 仅父模块可见
fn narrowest() {}         // 默认:仅当前模块及其子模块可见

// 对照 C:
// pub           ≈ 非 static 的全局符号(extern 默认)
// pub(crate)    ≈ 链接器内部符号(不导出到动态符号表)
// 默认私有      ≈ static 修饰的全局/函数

mod api {
    pub fn public_api() {}   // 依赖方可调用
    pub(crate) fn internal() {} // 库内部各模块可用,外部不可见
}

fn main() {
    api::public_api();
    api::internal();         // ✓ 同 crate 内可见
}
```

工程建议:pub(crate) 是"库内部共用"的标准答案

C 的"多文件共享但不想对外暴露"靠 static 与符号表技巧;Rust 直接写 `pub(crate) :crate` 内部随意调用,编译成库后外部无法触碰。生产库的 API 设计基线:能 `pub(crate)` 就不 `pub`——`pub` 的每一项都是永久承诺(第13.7节 `semver` 会讲为什么)。

§13.5 use 路径与重导出

13.5.1 路径的三种基准

```
// 绝对路径:从 crate 根开始
use crate::math::complex::norm;

// 相对路径:从当前模块开始
// use self::helper::x;    // self:: 当前模块
// use super::helper::x;   // super:: 父模块

mod helper {
    pub fn x() -> u32 { 1 }
}

mod app {
    // super 引用父模块(crate 根)里的 helper
    use super::helper::x;
    pub fn run() -> u32 { x() }
}

fn main() {
    println!("{}", app::run());
}
```

13.5.2 pub use:重导出,定义"公共门面"

```
// 库作者视角:内部组织 vs 对外门面
// src/lib.rs
mod internal {                                // internal 本身不公开
    pub struct Config { pub port: u16 }
    pub fn load() -> Config { Config { port: 8080 } }
}

// 但对外提供便捷入口:重导出(等价"转发引用",对照头文件的二次声明)
pub use internal::{Config, load};

// 使用方视角:use mylib::{Config, load}; 与库内部结构无关
// 好处:内部重构(改模块名/层级)不影响依赖方——门面稳定,内部自由
```

⚠ 易错点 2:use 与 pub use 的区别

use 只在本模块内绑定名字(私有); pub use 同时把它"加入本模块的公共 API"——依赖方可以 use 你重导出的名字,而无需知道它的真实来源。对照 C:pub use 相当于头文件里 typedef/声明了另一个文件里的符号。库设计惯例:对外只暴露 crate 根的门面(pub use 汇总),内部模块一律 pub(crate)。

§13.6 Cargo 全流程:构建、测试、依赖

13.6.1 常用命令速览

```
# 新建:二进制工程(默认)与库工程
cargo new myapp           # 生成 src/main.rs + Cargo.toml
cargo new --lib mylib      # 生成 src/lib.rs

# 构建与运行
cargo build                # 调试构建 → target/debug/
cargo build --release      # 发布构建 → target/release/
cargo run                  # 构建并运行
cargo check                # 只查类型错误,不生成代码(最快的体检)

# 测试与文档
cargo test                 # 运行全部测试(单元/集成/文档测试)
cargo doc --open           # 生成并打开 API 文档

# 依赖管理
cargo add serde             # 添加依赖(自动写进 Cargo.toml)
cargo update               # 按版本约束更新 Cargo.lock
cargo search json           # 在 crates.io 搜索

# 依赖树与审计
cargo tree                 # 查看依赖树
cargo audit                 # 检查已知漏洞(生产 CI 必备)
```

13.6.2 Cargo.toml 核心字段与版本约束

```
# ---- Cargo.toml(注释为教学说明)----
[package]
name = "myapp"
version = "0.1.0"           # 语义化版本:主.次.补丁
edition = "2021"           # 语言版本(第0章提过 2021 edition)
authors = ["you <you@example.com>"]
description = "示例工程"

[dependencies]
serde = { version = "1", features = ["derive"] } # 版本约束 ^1
rand = "0.8"           # ^0.8:允许 0.8.x,不允许 0.9
# 版本语义(semver):^1.2.3 允许 1.2.3 及以上、小于 2.0
# 0.x 系列:^0.8 只允许 0.8.x(0.8.1 是"破坏性"的,语义特殊)

[dev-dependencies]           # 仅测试用的依赖
pretty_assertions = "1"

[profile.release]           # 发布构建优化
opt-level = 3               # 对照 -O3
lto = true                  # 链接时优化
```


13.6.3 测试三形态

```
// 1) 单元测试:与被测代码同文件,#[cfg(test)] 只在测试构建编译
#[cfg(test)]
mod tests {
    use super::*;          // 导入被测模块的私有项(super 指向父模块)

    #[test]
    fn add_works() {
        assert_eq!(add(1, 2), 3);
    }

    #[test]
    fn add_handles_negative() {
        assert!(add(-1, 2) > 0);
    }
}

fn add(a: i32, b: i32) -> i32 { a + b }
```

// 2) 集成测试:tests/ 目录下的文件,以"外部使用方"视角调用库 API

```
// tests/api.rs:
// use mylib::load;
// #[test]
// fn loads_default() { assert_eq!(load().port, 8080); }
```

// 3) 文档测试:代码块里的示例被当作测试运行

```
// /// 计算两数之和
// /// ```
// /// assert_eq!(mylib::add(1, 2), 3);
// /// ```
// pub fn add(a: i32, b: i32) -> i32 { a + b }
```

§13.7 workspace 与 crates.io

13.7.1 workspace:多 crate 的单一工程

```
# ---- 根 Cargo.toml(仅声明成员,不写 [package])----
[workspace]
members = ["math-lib", "math-cli"]
resolver = "2"                # 现代依赖解析

# ---- 目录结构 ----
# myproj/
# └─ Cargo.toml                (workspace 声明)
# └─ math-lib/
#   └─ Cargo.toml
#   └─ src/lib.rs
# └─ math-cli/
#   └─ Cargo.toml              (依赖 math-lib,可用 path 或 workspace = true)
#   └─ src/main.rs

# ---- math-cli/Cargo.toml ----
# [dependencies]
# math-lib = { path = "../math-lib" }  # 路径依赖:本地开发

# 根目录统一操作:
# cargo build                  # 构建全部成员
# cargo test                   # 测试全部成员
# cargo run -p math-cli -- 参数  # 运行指定成员

# 多成员共享依赖:根 Cargo.toml 加 [workspace.dependencies]
# rand = "0.8"
# 成员里写:rand = { workspace = true }
```

13.7.2 crates.io:发布与使用

阶段	命令/操作	说明
注册账号	cargo login 令牌	crates.io 网站生成 API 令牌
检查打包	cargo package	本地验证 .crate 内容(会跑 cargo test)
发布	cargo publish	版本号不可重复,一经发布不可覆盖
使用	cargo add 包名	自动选择满足约束的最新版本
锁版本	Cargo.lock	应用工程提交锁文件,固定可复现构建
更新	cargo update	在版本约束内升级
文档	docs.rs 自动构建	发布即文档,与版本绑定

★ 重点:语义化版本与"pub 即承诺"

semver:主版本号变化 = 破坏性变更;次版本 = 新增功能(兼容);补丁 = 修复(兼容)。Rust 生态严格执行:你把 API 标成 pub 并发布 1.0 后,任何破坏性改动都要求升主版本——这就是"pub 是永久承诺"的原因。依赖方读 Cargo.toml 的 ^1.2 就能知道"补丁与次版本更新自动接受,主版本升级需手动"。生产上:锁文件 + CI 固定工具链 + cargo audit,是依赖管理的四件套。

⚠ 易错点 3:同名文件与同名目录的模块冲突

Rust 允许 src/math.rs 与 src/math/ 目录并存:math.rs 里的子模块文件放在 math/ 目录下。若两个位置都想定义"math"本身,会报"file for module found at both"错误。新项目统一采用"math.rs + math/ 子文件"的形态(\$13.3 形态二),别混用 mod.rs 老形态,避免二义;迁移老工程时先删 mod.rs 再搬文件。

依赖管理的完整生产姿势,是"四件套"的组合:① Cargo.lock 提交进版本库——固定每个依赖的确切版本,任何机器、任何时候构建结果一致;② 工具链固定——rust-toolchain.toml 文件钉死 rustc 版本,防止"昨天还编译,今天 rustup update 后挂了";③ cargo audit——在 CI 里跑,扫描依赖树的已知漏洞(CVE),对照 C 世界"依赖靠手工下载、漏洞靠新闻得知"的现状,这是生态级的安全升级;④ 语义化版本纪律——升级依赖前读 changelog,主版本升级(cargo update 不会自动做)要专门安排回归测试。这四个动作加起来,让"依赖"从"不可控的黑盒"变成"可审计、可复现、可升级"的工程资产。C 程序员把 Makefile 里的链接库路径换成 Cargo.toml,只是第一步;真正的迁移是接受"依赖树由工具管理"这一事实,把精力从"怎么链接"挪到"用哪个版本、何时升级"。

最后补一条关于文档的工程常识:docs.rs 为每个发布的 crate 版本自动构建 API 文档, cargo doc --open 生成本地文档。Rust 生态的文档是"随代码生成"的——给公共项写 /// 注释,文档、示例测试(doc test)、API 说明三者同步,不存在 C 世界"文档与头文件分家、头文件与实现分家"的三层漂移。进入生产后,阅读依赖代码的方式也因此改变:先读 docs.rs 的 API 页,再点进源码——这是 C 程序员需要主动养成的新习惯。

本章速查表

主题	结论 / 写法
模块声明	<code>mod math;</code> 找 <code>math.rs</code> ; <code>pub mod math;</code> 才对外可见
默认可见性	不写 <code>pub</code> 即私有(本模块及其子模块可见)
可见性四级	<code>pub</code> / <code>pub(crate)</code> / <code>pub(super)</code> / 默认私有
文件即模块	<code>main.rs/lib.rs</code> 是根; <code>mod x</code> 对应 <code>x.rs</code> 或 <code>x/mod.rs</code>
use 导入	<code>use crate::math::add;</code> <code>use super::*;</code> <code>use self::x;</code>
重导出	<code>pub use internal::{A, B};</code> 定义公共门面
别名/组	<code>use std::io::{Read, Write};</code> <code>use x as y;</code>
新建工程	<code>cargo new name(二进制)</code> / <code>cargo new --lib name(库)</code>
日常构建	<code>cargo build</code> / <code>cargo run</code> / <code>cargo check</code> (最快体检)
测试	<code>cargo test</code> ; 单元 <code>#[cfg(test)]</code> + 集成 <code>tests/</code> + 文档示例
加依赖	<code>cargo add serde</code> ; <code>Cargo.lock</code> 锁定可复现版本
版本约束	<code>^1.2</code> = <code>1.2</code> 至 <code><2.0</code> ; <code>0.x</code> 有特殊语义(<code>^0.8</code> = <code>0.8.x</code>)
workspace	<code>[workspace] members = [...]</code> ; 根目录统一 <code>build/test</code>
发布	<code>cargo login</code> → <code>cargo package</code> → <code>cargo publish</code> ; 版本不可覆盖
库 API 纪律	能 <code>pub(crate)</code> 就不 <code>pub</code> ; <code>pub</code> 是 semver 承诺

最小必会清单

- 能解释 Rust 模块系统如何解决头文件的三大痛点(守卫/顺序/污染)
- 能徒手把单文件拆成"根 + 两个模块文件"并正确声明
- 能说出 `pub` 与 `pub(crate)` 的差别,并知道库内部共享用哪个
- 能用 `super::` 与 `crate::` 写出正确的跨模块引用
- 能解释 `use` 与 `pub use` 的区别,并会用重导出搭建公共门面
- 能独立执行 `cargo new` → `add` → `build` → `test` → `run` 完整循环
- 知道单元测试(`#[cfg(test)]`)与集成测试(`tests/`)的分工
- 能读懂 `^1.2` 版本约束并解释 `Cargo.lock` 的作用
- 能搭建双成员 `workspace`(库 + 二进制)并用 `path` 依赖关联
- 知道 semver 与"pub 即承诺"对库 API 的约束

自测题

Q 1. C 的头文件守卫(`#ifndef`)在 Rust 里对应什么?为什么不需要?

✅ 参考答案

没有对应物。头文件守卫防的是"同一份声明被 `#include` 复制多次";Rust 的模块只编译一次,`mod` 声明是"引用"而非"复制"——同一个模块被多次 `use` 也只是建立引用,不会重复定义。同理,包含顺序问题也不存在:模块树的声明顺序与可见性由编译器统一检查,不依赖文本先后。

Q 2. 为什么 Rust 默认私有,而 C 默认全 `extern`?默认私有带来了什么?

✅ 参考答案

C 的符号默认全局可见,收窄靠 `static`——意味着"忘记收窄"就会污染全局命名空间,链接错误与符号冲突成为常态。Rust 默认私有,开放靠显式 `pub`——编译器强制你为每个对外符号做出决定;带来的直接收益:内部实现细节不可能意外泄漏,公共 API 边界由编译器检查,重构内部结构不影响依赖方(还有 `pub(crate)` 提供"内部共享"的中间档)。

Q 3. 一个库 `crate` 里,内部模块之间共享工具函数,该用什么可见性?

✅ 参考答案

`pub(crate)`:本 `crate` 内任意模块可调,编译为库后外部不可见。比 `pub` 安全(不给外部承诺)、比默认私有方便(不限制在单模块内)。C 的对应物:链接器内部符号或"声明在内部头文件、不进安装头"——但 C 没有语言级强制,Rust 是编译器保证。

Q 4. `use` 与 `pub use` 对依赖方有什么区别?

✅ 参考答案

`use` 是模块内的私有接线:依赖方看不到、也不该知道。`pub use` 把名字加入模块的公共 API:依赖方可以用你重导出的名字,且无需关心真实来源。库设计意义:内部把实现拆得再碎(模块层级随意调整),只要 `crate` 根的重导出不变,依赖方代码一行不改——`pub use` 是"内部自由重构"与"对外稳定门面"之间的桥梁。

Q 5. `Cargo.lock` 该不该提交进版本库?为什么?

✅ 参考答案

应用工程(二进制)必须提交:锁文件固定每个依赖的确切版本,保证任何机器、任何时候 `cargo build` 得到完全一致的依赖集(可复现构建,对照"供应商 vendoring"的效果)。库工程惯例不提交锁文件(依赖方会用自己的解析);但两者都受益于 CI 里固定工具链版本与 `cargo audit` 检查。C 世界的对照:没有锁文件,构建可复现性完全靠"团队口头约定版本"。

Q 6. 你在维护一个发布到 `crates.io` 的库。想给公共函数换签名,应该怎么做?

✅ 参考答案

按 `semver` 走:破坏性变更(签名、行为、删除 `pub` 项)必须升主版本号(`1.x` $\rightarrow$ `2.0`);若想温和过渡,标准流程是"弃用期":保留旧签名并标 `#[deprecated]`,新增新签名,一个次版本内共存,下个主版本再移除。因为 `pub` 即承诺:依赖方按 `^1.x` 解析,你的 `1.3` 里删掉函数会直接破坏他们的构建——这正是 Rust 生态依赖安全感的来源。

章末实验题:数学库 + CLI + 第三方依赖

实验 13:数学库 + CLI 工具工程

实验目标:搭建一个 workspace 工程:math-lib(库)提供基础数学运算与统计,math-cli(二进制)读取命令行参数调用库,并引入第三方依赖(rand)做随机练习生成。

实验要求:

- 任务 1:workspace 声明两个成员(覆盖:workspace §13.7)
- 任务 2:math-lib 库:模块拆分为 basic(加减乘除)与 stats(均值/方差),全部 pub 导出(覆盖:模块与文件 §13.3、重导出 §13.5)
- 任务 3:库内工具函数用 pub(crate),测试用 #[cfg(test)] 单元测试(覆盖:可见性 §13.4、测试 §13.6)
- 任务 4:math-cli 二进制依赖 math-lib(path 依赖),解析命令行执行"加/均/随机练习"(覆盖:依赖 §13.6)
- 任务 5:cargo add rand,用 rand 生成随机算术练习并自动判分(覆盖:第三方依赖 §13.6)
- 加分:为库写集成测试 tests/api.rs;用 cargo doc --open 生成文档

运行示例: cargo run -p math-cli -- add 3 5 → 8; cargo run -p math-cli -- avg 1 2 3 4 → 2.5; cargo run -p math-cli -- quiz → 随机题目与判分。

第一步 **一步:**建目录 math-ws,写根 Cargo.toml 声明 workspace;cargo new --lib math-lib 与 cargo new math-cli

第二步 **二步:**在 math-lib 建 src/basic.rs 与 src/stats.rs,lib.rs 声明并重导出

第三步 **三步:**写单元测试,cargo test -p math-lib 全绿

第四步 **四步:**math-cli 加 path 依赖,写 main:match 子命令分发

第五步 **五步:**cargo add rand,实现 quiz 子命令;cargo run -p math-cli 验证三种用法

参考答案要点

```
// ===== 实验 13:数学库 + CLI 工程 =====
// 覆盖知识点:workspace(§13.7)、模块与文件(§13.3)、重导出(§13.5)、
// 可见性(§13.4)、测试(§13.6)、依赖管理(§13.6)

// ---- 根 Cargo.toml ----
// [workspace]
// members = ["math-lib", "math-cli"]
// resolver = "2"

// ---- math-lib/Cargo.toml ----
// [package] name = "math-lib" version = "0.1.0" edition = "2021"

// ---- math-lib/src/lib.rs(库根:声明 + 重导出门面)----
pub mod basic;           // 模块文件 src/basic.rs
pub mod stats;           // src/stats.rs
// 门面重导出:依赖方 use math_lib::{add, mean};
pub use basic::{add, div, mul, sub};
pub use stats::{mean, variance};

// ---- math-lib/src/basic.rs ----
/// 加法
```

```

pub fn add(a: i64, b: i64) -> i64 { a + b }
pub fn sub(a: i64, b: i64) -> i64 { a - b }
pub fn mul(a: i64, b: i64) -> i64 { a * b }

/// 除法:除零返回 None(错误进类型,而非崩溃)
pub fn div(a: i64, b: i64) -> Option<i64> {
    if b == 0 { None } else { Some(a / b) }
}

// ---- math-lib/src/stats.rs ----
/// 均值:f64 向量
pub fn mean(xs: &[f64]) -> f64 {
    if xs.is_empty() { 0.0 } else { xs.iter().sum::<f64>() / xs.len() as f64 }
}

/// 方差:与均值的平方偏差的平均
pub fn variance(xs: &[f64]) -> f64 {
    if xs.is_empty() { return 0.0; }
    let m = mean(xs);
    xs.iter().map(|x| (x - m) * (x - m)).sum::<f64>() / xs.len() as f64
}

// ---- math-lib 单元测试:与被测代码同文件 ----
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn add_works() { assert_eq!(add(2, 3), 5); }

    #[test]
    fn div_by_zero_is_none() { assert_eq!(div(1, 0), None); }

    #[test]
    fn mean_and_variance() {
        let xs = [2.0, 4.0, 4.0, 4.0, 5.0, 5.0, 7.0, 9.0];
        assert!((mean(&xs) - 5.0).abs() < 1e-9);
        assert!((variance(&xs) - 4.0).abs() < 1e-9);
    }
}

// ---- math-lib/tests/api.rs(集成测试:外部视角)----
use math_lib::{add, mean};

#[test]
fn public_api_works() {
    assert_eq!(add(1, 1), 2);
    assert!((mean(&[1.0, 3.0]) - 2.0).abs() < 1e-9);
}

// ---- math-cli/Cargo.toml ----
// [package] name = "math-cli" version = "0.1.0" edition = "2021"
// [dependencies]
// math-lib = { path = "../math-lib" }
// rand = "0.8"

```

```
// ---- math-cli/src/main.rs ----
use rand::Rng;

fn print_usage(prog: &str) {
    eprintln!("用法:");
    eprintln!(" {prog} add <a> <b>      加法");
    eprintln!(" {prog} avg <数...>      均值");
    eprintln!(" {prog} quiz                随机练习");
}

fn main() {
    let args: Vec<String> = std::env::args().collect();
    let prog = args[0].as_str();
    let Some(cmd) = args.get(1).map(String::as_str) else {
        print_usage(prog);
        std::process::exit(2);
    };

    match cmd {
        "add" => {
            let a: i64 = args[2].parse().expect("数字");
            let b: i64 = args[3].parse().expect("数字");
            println!("{}", a + b, math_lib::add(a, b));
        }
        "avg" => {
            let xs: Vec<f64> = args[2..]
                .iter().map(|s| s.parse().expect("数字")).collect();
            println!("均值:{}", math_lib::mean(&xs));
        }
        "quiz" => {
            let mut rng = rand::thread_rng();
            let mut score = 0u32;
            for _ in 0..5 {
                let a: i32 = rng.gen_range(1..=20);
                let b: i32 = rng.gen_range(1..=20);
                let op = rng.gen_range(0..2); // 0=+, 1=-
                let (question, answer) = if op == 0 {
                    (format!("{a} + {b}"), a + b)
                } else {
                    (format!("{a} - {b}"), a - b)
                };
                println!("第 {} 题:{question} = ?", score + 1);
                let mut input = String::new();
                std::io::stdin().read_line(&mut input).unwrap();
                if input.trim() == answer.to_string() {
                    score += 1;
                    println!("✓ 正确");
                } else {
                    println!("✗ 答案是 {answer}");
                }
            }
            println!("得分:{score}/5");
        }
        other => {

```



```
        eprintln!("未知命令:{other}");
        print_usage(prog);
        std::process::exit(2);
    }
}
```

设计说明:本实验的骨架就是生产小工具的完整形态——库与二进制分离(逻辑可测试、可复用)、门面重导出(内部结构自由调整)、pub(crate) 防泄漏、测试三件套、第三方依赖一句话接入。注意两个细节:div 返回 Option 而非崩溃(第9章"错误进类型");let-else(第3章)处理缺失子命令。进阶练习:把 quiz 的输入解析换成 Result(非法输入不panic)、给 math-lib 加 #[deny(missing_docs)] 并用 cargo doc 验证——那时你就拥有一个"发布级"库的完整姿势了。

下一章预告:第14章 类型系统进阶——Option 空安全、enum 和类型、newtype 模式、类型推导边界与 impl Trait 返回;全书的收官之章。